

Class File Format and Bytecode Verification

Christopher League

JVM Seminar

20 October 1999

Some unresolved issues

- ◇ Code attribute
 - ▷ *Forbidden* if native or abstract
 - ▷ *Required* otherwise
- ◇ InnerClasses and Synthetic attributes

Attributes defined in JVM spec

	Class	Field	Method	Code
ConstantValue		★		
Code			★	
Exceptions			★	
InnerClasses	★			
Synthetic	★	★	★	
SourceFile	★			
LineNumberTable				★
LocalVariableTable				★
Deprecated	★	★	★	

Attributes for inner classes

- ◇ InnerClasses—for each class C in CP that is *not a member of a package*:
 - ▷ inner class info
 - ▷ outer class info, if C is a member
 - ▷ original name, unless anonymous
 - ▷ access flags (unchecked)
- ◇ Synthetic—marks fields and methods added by compiler (to implement scope)
- ◇ Both *required* when applicable

Inner class confusion

[LY99] refers to *Inner Classes Spec.* (online)

Q: How do inner classes affect the organization of the JVM?

- ◇ “no changes to class file format”
- ◇ “new features are implemented by the compiler”

Q: What are new requirements for Java 1.1 classes?

- ◇ “bytecodes must contain InnerClasses”
- ◇ “compilers required to use Synthetic”

Why?

Outline—Part 2

1. Class file layout

2. Verification

(a) Summary of constraints

(b) How and when they are verified

(c) Tracking uninitialized instances

(d) Checking exception handlers and subroutines

Categories of constraints

- ◇ **Static** constraints (§4.8.1)
 - ▷ well-formedness of the file
 - ▷ layout of instructions in code array
 - ▷ operands of each instruction
- ◇ **Structural** constraints (§4.8.2)
 - ▷ relationships between instructions

Recall—Code attribute

The Code attribute contains:

- ◇ *Max stack*
- ◇ *Max locals*
- ◇ *Byte code array*
- ◇ *Exception table* (start pc, end pc, handler pc)*
- ◇ *Attributes* (debug info)

Examples of static code constraints

- ◇ *Jump targets are within code array*
- ◇ *Constant pool indices are type-correct*
 - ▷ *new must reference a (non-abstract) Class*
 - ▷ *invoke* must reference a MethodRef*
- ◇ *Local variable refs within bounds*
- ◇ **switch layout*

Examples of structural constraints

Regardless of execution path:

- ◇ *type-correct use of local variables and stack operands*
- ◇ *stack size always between 0 and max_stack*
- ◇ *<init> called only on uninitialized instances*
- ◇ *method and field access only on initialized instances*
- ◇ *<init> must call super before accessing fields*
- ◇ *invoke* arguments match type descriptor*
- ◇ *atomic treatment of long and double*

Operational view of constraints

- ◇ **Pass 1:** well-formedness of the file
- ◇ **Pass 2:** static constraints outside code array
 - ▷ `final` checking
 - ▷ constant pool constraints
- ◇ **Pass 3:** data-flow analysis on code array
 - ▷ at each point, stack has unique type
 - ▷ locals accessed with appropriate type
 - ▷ type-check method and field usage
- ◇ **Pass 4:** type-check **external** field and method refs
 - ▷ deferred to avoid loading classes prematurely

Pass 3: data-flow

Spec gives the detailed **algorithm!**

1. Initial state:

- ◇ Local variables have types indicated by method's type descriptor
- ◇ Excess local variables are **uninitialized**
- ◇ Operand stack is empty

2. Model the instruction's effect (and type check!)

3. Determine successor instructions

4. Merge new state into successor instructions

5. Iterate...

Pass 3: data flow (step 2)

1. Initial state
2. Model the instruction's effect (and type check!)
 - ◇ check operand stack types, underflow
 - ◇ check local variable types
 - ◇ if instruction pushes, check overflow and update stack type
 - ◇ if instruction modifies local, record new type
3. Determine successor instructions
4. Merge new state into successor instructions
5. Iterate...

Pass 3: data flow (step 3)

1. Initial state
2. Model the instruction's effect (and type check!)
3. Determine successor instructions
 - ◇ next instruction (unless goto, return, athrow)
 - ◇ targets of branch
 - ◇ any applicable **exception handlers**
4. Merge new state into successor instructions
5. Iterate...

Pass 3: data flow (step 4)

1. Initial state
2. Model the instruction's effect (and type check!)
3. Determine successor instructions
4. Merge new state into successor instructions
 - ◇ stacks must be same size
 - ◇ stacks must have same types
 - ▷ except **reference**—use first common superclass
 - ▷ (**Q**: does this force loading of all superclasses?)
 - ◇ locals with incompatible types become **unusable**
5. Iterate...

Observations about data flow

- ◇ **Definite assignment** in JLS not directly related
 - ▷ Local variables never declared, just initialized
 - ▷ Can be reused with different types
- ◇ **Exception table**
 - ▷ Protected regions need not be properly nested
 - ▷ A handler can protect itself
 - ▷ Regular code can fall through into handler
- ◇ Discussion: how good is this algorithm as a **spec**?

Tracking uninitialized instances

Recall these constraints:

- ◇ `<init>` called only on uninitialized instances
- ◇ method and field access only on initialized instances

Unlike in Java, *allocation* is separate from *initialization*.

```
Point p = new Point(3);
```

```
p.Print();
```

Tracking uninitialized instances

Recall these constraints:

- ◇ `<init>` called only on uninitialized instances
- ◇ method and field access only on initialized instances

Unlike in Java, *allocation* is separate from *initialization*.

```
Point p = new Point(3);    new #1
                           dup
                           iconst_3
                           invokespecial #4
p.Print();                 invokevirtual #5
```

Stack types during object creation

First attempt:

Instruction	New stack type
—	[]
new #1	[Point ⁰]
dup	[Point ⁰ , Point ⁰]
iconst_3	[Int, Point ⁰ , Point ⁰]
invokespecial #4	[Point ¹]
invokevirtual #5	[]

Magic!

Stack types during object creation

With a trickier program:

Instruction	New stack type
—	[]
new #1	[Point ⁰]
dup	[Point ⁰ , Point ⁰]
new #1	[Point ⁰ , Point ⁰ , Point ⁰]
swap	[Point ⁰ , Point ⁰ , Point ⁰]
iconst_3	[Int, Point ⁰ , Point ⁰ , Point ⁰]
invokespecial #4	[Point ⁰ , Point ¹]
invokevirtual #5	✗

Stack types during object creation

With a trickier program:

Alias analysis

Instruction	New stack type
—	[]
new #1	[Point ⁰]
dup	[Point ⁰ , Point ⁰]
new #1	[Point ⁰ , Point ⁰ , Point ⁰]
swap	[Point ⁰ , Point ⁰ , Point ⁰]
iconst_3	[Int, Point ⁰ , Point ⁰ , Point ⁰]
invokespecial #4	[Point ⁰ , Point ¹]
invokevirtual #5	✗

Avoiding alias analysis

1. Uninitialized type indicates **creating instruction**
2. On initialization, change all occurrences of that type.

—	[]
1: new #1	[Point ₁]
2: dup	[Point ₁ , Point ₁]
3: iconst_3	[Int, Point ₁ , Point ₁]
4: invokespecial #4	[Point]
5: invokevirtual #5	[]

3. **Prevent backwards branch** while stack/locals contain uninitialized object.

Verifying initialization methods

JLS and JVM agree on this constraint:

- ◇ *<init> must call super <init> before accessing fields and methods*

On entry to instance initialization method, local variable 0 has type of uninitialized instance of current class.

After another <init> is invoked, all occurrences of this special type become normal class type.

Subroutines

◇ `jsr <label>`

Transfer control, pushing return pc onto the operand stack.

◇ `ret <local>`

Return to address stored in local variable.

```
10: jsr 13
```

```
11: ...
```

```
13: astore_3
```

```
14: ret 3
```

“This asymmetry is intentional.”

Subroutines—How are they used?

```
int bar(int i) {  
    try {  
        if (i == 3) return this.foo();  
    } finally {  
        this.ladida();  
    }  
    return i;  
}
```

3 ways to enter finally block.

Subroutines—Why are they hard?

Before, when merging local variable states,

- ◇ locals with incompatible types became **unusable**

Must relax this; locals not accessed by the subroutine may stay as they are.

Also, need to track return addresses to verify LIFO behavior.

